

MAKERERE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
SCHOOL OF COMPUTING & INFORMATICS TECHNOLOGY

PROGRAMME: BACHELOR OF COMPUTER SCIENCE
COURSE NAME: SOFTWARE DEVELOPMENT PROJECT
COURSE CODE: CSC 1202

DATE: 15th May 2025

TIME: 8:00 AM To 11:00 AM

EXAMINATION INSTRUCTIONS

- 1. ATTEMPT ANY FIVE (5) QUESTIONS.**
- 2. USE THE CASE STUDY PROVIDED TO ANSWER THE QUESTIONS.**
- 3. IN CASES WHERE CODE HAS BEEN PROVIDED, DO NOT REWRITE CODE, PROVIDE THE MISSING PARTS WITH CORRECT NUMBER LABELS.**
- 4. DO NOT OPEN THIS EXAM UNTIL YOU ARE TOLD TO DO SO.**
- 5. ALL ROUGH WORK SHOULD BE IN YOUR ANSWER BOOKLET.**

Case Study Description: Academic Issue Tracking System

Makerere University has commissioned your team to develop the **Academic Issue Tracking System (AITS)**. The system allows students to log academic issues such as missing marks, correction requests, and appeals. Key roles include **Students**, **Lecturers**, and the **Academic Registrar**.

Each user interacts with a personalized dashboard:

- **Students** can log issues, view status, and receive notifications.
- **Lecturers** can view issues assigned to them and update resolution status.
- **Academic Registrars** manage and assign issues to appropriate staff.

The system uses **Django (backend)**, **React (frontend)**, and is deployed on **Heroku**. APIs are built using Django REST Framework. Real-time updates are sent via email or toast notifications. The development follows Agile practices in teams using GitHub. Use the case above to answer the following questions:

Question 1

In the AITS system, students report academic issues. Each issue is submitted by a student and may be assigned to a lecturer. Both students and lecturers belong to departments.

- a) Explain how ForeignKey relationships are used in Django to represent the following in the AITS system: **(4 Marks)**
- Each issue is submitted by a student.
 - Each student and lecturer belongs to a department.
- b) The code below defines models for Department, User, and Issue. However, lines 3, 7, and 12 are missing important details. Fill in the missing parts to complete the model relationships. **(10 Marks)**

```
1 | class Department(models.Model):
2 |     name = models.CharField(max_length=100)
3 |     # Add __str__ method here
4 | class User(AbstractUser):
5 |     is_student = models.BooleanField(default=False)
6 |     is_lecturer = models.BooleanField(default=False)
7 |     # Add relationship to Department
8 | class Issue(models.Model):
9 |     title = models.CharField(max_length=100)
10 |     description = models.TextField()
11 |     status = models.CharField(max_length=20, default="Pending")
12 |     # Add relationship to student
13 |     assigned_to = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True,
related_name="assigned_issues")
```

- c) Write a Django query to retrieve all unresolved issues (i.e., status not "Resolved") that were submitted by students in the same department as the currently logged-in registrar (request.user). **(6 Marks)**

Question 2

As a frontend developer working on the AITS system, you are responsible for building pages where students can view the academic issues they submitted. These are retrieved from a backend REST API.

- a) Using an appropriate diagram, explain a REST API and describe how it helps frontend developers build features like the student issue list in the AITS system. **(4 Marks)**
- b) You have been provided with the following API:
- What HTTP method and URL should you use to call this API? **(2 marks)**
 - What is the expected response format? **(2 marks)**
 - How would you identify which issues are still pending in the frontend? **(2 marks)**

GET /api/student-issues/

Description: Returns a list of issues submitted by the currently logged-in student.

Response Example:

```
[
  {
    "id": 1,
    "title": "Missing marks",
    "status": "Resolved",
    "category": "Appeal"
  },
  {
    "id": 2,
    "title": "Course not added",
    "status": "Pending",
    "category": "Registration"
  }
]
```

- c) The following React component is supposed to fetch the list of student issues from the API and display them in a table. However, lines 2, 5, and 9 are incorrect or missing. Fix the missing parts to complete the component. **(10 Marks)**

```
1 | function StudentIssueTable() {
2 |   const [issues, setIssues] = useEffect([]);
3 |
4 |   useEffect(() => {
5 |     fetch('_____')
6 |       .then(res => res.json())
7 |       .then(data => setIssues(data));
8 |   }, []);
9 |   return issues.map(issue => (
10 |     <tr>
11 |       <td>{issue.title}</td>
12 |       <td>{issue.status}</td>
13 |       <td>{issue.category}</td>
14 |     </tr>
15 |   ));
16 | }
```

Question 3

In the AITS system, students can submit academic issues using a form on the frontend. The React component must handle form inputs, send data to the backend API, and provide feedback to the user.

- a) Explain how `useState` and the `onChange` event handler are used together to manage form input in React. Why is this approach important when building interactive forms? **(4 Marks)**
- b) Below is a React component that is incomplete. It is supposed to:
 - i. Track user input in the form fields, **(2 Marks)**
 - ii. Submit the form to the API endpoint `/api/issues/`, **(4 Marks)**
 - iii. Display a success message on submission. **(4 Marks)**

However, lines 3, 6, and 11 are either missing or incorrect. Fill in the missing parts of the code below:

- c) Modify the form component so that a "Submitting..." message is displayed while the form is being submitted. Briefly describe what extra state you would add, and where you would update it in the `handleSubmit` function. **(6 Marks)**

```

1 | function IssueForm() {
2 |   const [formData, setFormData] = useState({ title: "", category: "", description: "" });
3 |   const [success, setSuccess] = useState();
4 |
5 |   const handleChange = (e) => {
6 |     _____
7 |   };
8 |
9 |   const handleSubmit = async (e) => {
10 |     e.preventDefault();
11 |     const res = await fetch('/api/issues/', {
12 |       method: "POST",
13 |       headers: { 'Content-Type': 'application/json' },
14 |       body: JSON.stringify(formData)
15 |     });
16 |     if (res.ok) {
17 |       _____
18 |     }
19 |   };
20 |
21 |   return (
22 |     <form onSubmit={handleSubmit}>
23 |       <input name="title" onChange={handleChange} value={formData.title} />
24 |       <input name="category" onChange={handleChange} value={formData.category} />
25 |       <textarea name="description" onChange={handleChange} value={formData.description}></textarea>
26 |       <button type="submit">Submit</button>
27 |       {success && <p>Issue submitted successfully!</p>}
28 |     </form>
29 |   );
30 | }

```

Question 4

- a) The following Django signal is intended to send an email when an issue's status changes to "Resolved". However, the implementation has bugs in lines 2, 3, and 8. Correct only the marked lines so that the signal works as expected and only sends email when an existing issue is updated to "Resolved". **(12 Marks)**

```

1 | @receiver(post_save, sender=Issue)
2 | def notify_on_resolve(instance, created, **kwargs):
3 |     if instance.status == "Resolved":
4 |         send_mail(
5 |             "Issue Resolved",
6 |             f"The issue {instance.title} has been resolved.",
7 |             "admin@aits.com",
8 |             instance.student.email
9 |         )

```

- b) The React code below is meant to display a toast notification when an issue update API response shows "Resolved" as the status. However, there are bugs in lines 1, 4, and 5. Identify and fix only the incorrect lines so that the toast displays correctly when the status is "Resolved". **(8 Marks)**

```
1 | const updateIssue = async () => {  
2 |   const res = await fetch('/api/issues/5/', { method: 'PATCH' });  
3 |   const data = await res.json();  
4 |   if (data.status == "Resolved") {  
5 |     Toast("Resolved!");  
6 |   }  
7 | };
```

Question 5

- a) Describe the role of an API in the AITS project, focusing on how it enables interaction between the React frontend and the Django backend. **(4 Marks)**
- b) You are building an endpoint for the AITS system where students can view the list of issues they have submitted. Complete the missing lines to create a Django REST Framework view that handles a GET request to /api/student-issues/, returning issues for the currently logged-in student. The code below has gaps in lines 3, 5, and 7. Fill them correctly and add a comment on what your code does (format: Line 3| modified code # comment). **(6 Marks)**

```
1 | from rest_framework.views import APIView  
2 | from rest_framework.response import Response  
3 | from .models import _____  
4 | from .serializers import IssueSerializer  
5 | class StudentIssuesView(_____):  
6 |     def get(self, request):  
7 |         issues = Issue.objects.filter(_____)  
8 |         serializer = IssueSerializer(issues, many=True)  
9 |         return Response(serializer.data)
```

- c) You are now implementing the frontend to fetch the student's issues and display them on the dashboard. Below is a React component that is supposed to:
- Fetch the student's issues from /api/student-issues/
 - Display the list in an unordered list

The code below has mistakes in lines 2, 5, and 10. Correct them and add a comment on what your code does (format: Line 3| modified code # comment). **(10 Marks)**

```
1 | function IssueDashboard() {
2 |   const [issues, setIssues] = useEffect([]);
3 |
4 |   useEffect(() => {
5 |     fetch('/api/_____')
6 |       .then(response => response)
7 |       .then(data => setIssues(data));
8 |   }, []);
9 |
10 |   {issues.map(issue => (
11 |     <li key={issue.id}>{issue.title} - {issue.status}</li>
12 |   ))}
13 | }
```

Question 6

You are deploying the AITS application to **Heroku**, a cloud platform for hosting web applications. Correct configuration is critical to ensure the application runs securely and efficiently.

- List two important settings that must be configured in Django when deploying to Heroku and explain why each is necessary. **(4 Marks)**
- The following Django settings file snippet is incomplete or incorrect. Carefully review the numbered lines and fix the mistakes in lines 2, 4, and 7. **(8 Marks)**

```
1 | import os
2 | DEBUG = True
3 |
4 | ALLOWED_HOSTS = []
5 |
6 | DATABASES = {
7 |   'default': {
8 |     'ENGINE': 'django.db.backends.sqlite3',
9 |     'NAME': BASE_DIR / 'db.sqlite3',
10 |   }
11 | }
12 |
13 | STATIC_ROOT = "
```

- c) Below are the deployment files used to deploy AITS on Heroku. There are mistakes in Procfile (line 2) and environment variables setup (line 5). Fix the mistakes in the following numbered lines: **(8 Marks)**

Procfile

```
1 | web: gunicorn aits.wsgi
2 | (missing settings here)
```

.env File (Environment Variables)

```
3 | SECRET_KEY=mysecretkey
4 | DEBUG=True
5 | ALLOWED_HOSTS=*
```

Question 7

In the AITS system, students submit academic issues using a web form. The backend must validate this data, and the frontend should show friendly error messages when something goes wrong.

- a) Explain the importance of input validation in backend APIs and why error feedback is important in frontend user interfaces. **(4 Marks)**
- b) Below is a Django REST Framework serializer for the Issue model. It is supposed to ensure that both title and category fields are required. However, there are missing lines in 3, 5, and 6. Fill in the missing code for these lines to implement proper validation. **(8 Marks)**

```
1 | from rest_framework import serializers
2 | from .models import Issue
3 | class ____ (serializers.ModelSerializer):
4 |     class Meta:
5 |         model = ____
6 |         fields = ____
7 |
8 |     def validate(self, data):
9 |         if not data.get('title') or not data.get('category'):
10 |             raise serializers.ValidationError("Title and category are required.")
11 |         return data
```

- c) Below is a React form for submitting issues. It works correctly when the submission is successful but fails silently when the server returns an error. Fix lines 3, 5, and 8 to: **(8 Marks)**
- Catch API errors,

- Store the error message,
- Display it below the form.

```
1 | function IssueForm() {  
2 |   const [formData, setFormData] = useState({ title: "", category: "", description: "" });  
3 |   const [error, setError] = useState();  
4 |  
5 |   const handleSubmit = async (e) => {  
6 |     e.preventDefault();  
7 |     const res = await fetch('/api/issues/', {  
8 |       method: "POST",  
9 |       headers: { 'Content-Type': 'application/json' },  
10 |      body: JSON.stringify(formData)  
11 |    });  
12 |    if (!res.ok) {  
13 |      // handle error  
14 |    }  
15 |  };  
16 |  
17 |  return (  
18 |    <form onSubmit={handleSubmit}>  
19 |      <input name="title" value={formData.title} />  
20 |      <input name="category" value={formData.category} />  
21 |      <textarea name="description" value={formData.description} />  
22 |      <button type="submit">Submit</button>  
23 |      {error && <p className="error">{error}</p>}  
24 |    </form>  
25 |  );  
26 | }
```

Question 8

The AITS system must be thoroughly tested to ensure it works as expected. You are asked to implement and fix Django unit tests related to the Issue model and API.

- a) Explain why automated testing is important in backend development and give one reason why a test might fail even if the code seems correct. **(4 Marks)**
- b) You are asked to test whether a logged-in student can successfully create an issue through the API. Write a Django test case method that: **(8 marks)**
 - Creates a test user,
 - Logs in the user,
 - Submits a POST request to /api/issues/ with valid issue data,
 - Asserts that the response status code is 201.
- c) The test below is meant to check that a student cannot view all issues, but it is failing. Review the numbered lines and fix the errors in lines 2, 5, and 6. **(8 Marks)**

```
1 | class IssueAccessTest(TestCase):
2 |     def test_view_all_issues(self):
3 |         student = User.objects.create_user(username="test", password="pass123")
4 |         self.client.login(username="test", password="pass123")
5 |         response = self.client.get('/api/issues/')
6 |         self.assertEqual(response.status_code, 200)
```